Fluimos con la precisión: implementación de AquaSenseCR bajo una arquitectura NoSQL de columnas anchas mediante ScyllaDB

David Brenes, Fernando López, Catalina Monge y Tatiana Ramos

Universidad de Costa Rica
Facultad de Ciencias Económicas
Escuela de Estadística
Ciudad Universitaria Rodrigo Facio, Montes de Oca, San José, Costa Rica

***Resumen*—Este trabajo presenta AquaSenseCR, una solución académica para almacenar y consultar mediciones sintéticas de una red de agua potable mediante ScyllaDB y el modelo NoSQL de columnas anchas. Se generaron de manera reproducible 1 000 000 de lecturas correspondientes a 1000 sensores distribuidos en 20 zonas, con un 1 % de anomalías de presión. El diseño comprende cuatro tablas orientadas a consultas por sensor, zona, anomalías y resúmenes diarios. La carga de las dos tablas principales requirió 2 000 000 de escrituras y se completó en 199,04 segundos. Además, se evaluaron cinco consultas mediante 30 repeticiones y cinco ejecuciones de calentamiento, con medianas de latencia entre 0,875 y 4,294 ms. Las verificaciones de registros y una comparación exploratoria de escrituras concurrentes complementaron la evaluación. Los resultados respaldan el funcionamiento del diseño para el volumen ensayado en un entorno local de un solo nodo.**

***Palabras clave*—ScyllaDB, NoSQL, columnas anchas, agua potable, particionamiento, telemetría, rendimiento.**

I. Introducción y problema

En el contexto contemporáneo, la gestión de bases de datos ha estado fuertemente influenciada por el modelo relacional y por el uso de *Structured Query Language* (SQL) como lenguaje de consulta y manipulación de información. Como se ha estudiado a lo largo del curso [1], el desarrollo de herramientas que permitieran almacenar y recuperar datos mediante instrucciones comprensibles representó un hito para este campo. No obstante, la creciente diversidad de aplicaciones y volúmenes de información ha planteado retos que motivaron el desarrollo de modelos NoSQL, los cuales se caracterizan por formas alternativas de organizar los datos [5].

Un ejemplo de estas necesidades es el monitoreo de redes de agua potable, donde se recopilan observaciones continuas de variables como caudal, presión, temperatura y calidad. Más allá de los dispositivos utilizados para registrar dichos atributos, el alcance de esta información depende también de su ubicación, almacenamiento y recuperación oportuna. En este sentido, Ostfeld et al. [2] muestran la relevancia de las redes de sensores y de los procedimientos de análisis para estudiar sistemas de distribución de agua.

La acumulación de mediciones, por sí sola, no garantiza que estas puedan aprovecharse. También es necesario responder interrogantes concretas, como recuperar las lecturas recientes de un dispositivo, consultar periodos por zona, localizar comportamientos anómalos u obtener indicadores diarios. Estas preguntas incorporan dos dimensiones que no pueden desligarse: la ubicación de las observaciones y el momento en que fueron registradas. Un historial de presión, por ejemplo, puede tener interpretaciones diferentes según el sensor consultado y el intervalo temporal seleccionado. Aunque estas operaciones pueden resolverse mediante sistemas relacionales, las aplicaciones de telemetría requieren examinar cuidadosamente la organización de sus registros y los patrones de acceso que predominan.

En este sentido, conocer las particularidades del problema resulta fundamental para determinar qué alternativa de almacenamiento conviene utilizar. Los modelos NoSQL pueden responder a necesidades de escalabilidad y manejo de grandes volúmenes de información, pero su aplicación supone decisiones relacionadas con la consistencia, la distribución y las consultas previstas [3]. Por tanto, no se trata de asumir una falsa dicotomía donde un modelo sea automáticamente superior al otro, sino de reconocer los compromisos que implica cada propuesta [4].

Con el propósito de explorar estas decisiones se desarrolló el caso de la empresa ficticia **AquaSenseCR**, dedicada al monitoreo de agua potable mediante sensores distribuidos geográficamente. El problema central consiste en almacenar un historial creciente de mediciones y recuperarlo por sensor, zona e intervalo temporal, sin depender de recorridos completos de las tablas. También se requiere identificar anomalías y consultar indicadores operativos previamente calculados. Para atender estas necesidades se seleccionó el modelo NoSQL de columnas anchas o *wide-column*, implementado con ScyllaDB.

En este contexto, el objetivo general consiste en implementar y evaluar una solución reproducible para almacenar un millón de mediciones sintéticas y analizar el rendimiento de su recuperación. Específicamente, se busca: (i) justificar el diseño de tablas y claves; (ii) verificar las consultas por sensor, zona y periodo; (iii) recuperar anomalías y resúmenes diarios; (iv) examinar la organización mensual de los registros y comparar

dos patrones de escrituras concurrentes; y (v) documentar los tiempos de carga, las latencias y el entorno de ejecución.

II. Fundamentos teóricos del modelo NoSQL

II-A. Clasificación y características de los modelos NoSQL

El universo de los modelos NoSQL comprende sistemas heterogéneos que proponen distintas formas de almacenar y recuperar información. Entre sus familias más conocidas se encuentran los modelos clave-valor, documentales, de grafos y de columnas anchas [5]. Si bien comparten ciertas características relacionadas con el volumen y la organización de los datos, no constituyen tecnologías equivalentes ni ofrecen las mismas posibilidades de consulta.

En consecuencia, la elección de un modelo debe establecerse en diálogo con las necesidades de cada aplicación. Cattell [4] muestra que estas alternativas presentan compromisos distintos en términos de escalabilidad, consistencia y mecanismos de almacenamiento. Por ejemplo, una base documental puede resultar conveniente cuando la información se representa mediante documentos de estructura variable, mientras que un modelo de grafos adquiere relevancia cuando predominan las relaciones y recorridos entre entidades. Estos ejemplos permiten comprender por qué la denominación NoSQL no describe una solución única.

Por esa razón, identificar la familia tecnológica constituye apenas el primer paso: también debe examinarse cómo se construyen sus claves, qué consultas pueden localizar directamente los datos y cuáles exigen operaciones adicionales. Esta distinción adquiere mayor relevancia cuando el volumen de observaciones aumenta porque pequeñas decisiones de organización pueden repetirse sobre millones de registros.

Para AquaSenseCR interesa particularmente el modelo de columnas anchas, pues permite agrupar registros relacionados a partir de claves y organizar su recuperación según patrones de acceso definidos. El interés no radica en abandonar los principios relacionales por sí mismos, sino en estudiar una alternativa cuya organización física pueda responder a consultas temporales conocidas de antemano.

II-B. Organización de los datos en columnas anchas

Uno de los antecedentes centrales de este modelo se encuentra en Bigtable, desarrollado por Chang et al. [6], donde los datos se identifican mediante claves de fila, columnas y marcas temporales. Este planteamiento permite organizar grandes conjuntos de registros de acuerdo con criterios que favorecen su localización y tratamiento.

A diferencia de un diseño relacional enfocado principalmente en representar entidades y sus relaciones, las bases de columnas anchas otorgan una importancia particular a la agrupación de los registros y a la manera en que serán consultados. Las claves no se limitan a identificar información: también intervienen en su organización y, dependiendo del sistema, en su distribución.

Esta diferencia adquiere especial importancia cuando se almacenan observaciones continuas de numerosos dispositivos, pues no siempre interesa recuperar una medición aislada, sino estudiar la evolución de los valores registrado. Para un historial de sensores, esta distinción ayuda a comprender por qué una consulta que requiere las últimas lecturas de un dispositivo puede organizarse a partir de su identificador y de una referencia temporal.

II-C. Modelado orientado a patrones de consulta

Una característica fundamental de estos sistemas es que las consultas previstas deben considerarse desde las primeras etapas de diseño. Esto no significa que el rendimiento sea ajeno a las bases relacionales, sino que, en las columnas anchas, la elección de claves y agrupaciones condiciona directamente las operaciones que pueden ejecutarse eficientemente. Sadalage y Fowler [7] destacan la importancia de adecuar la estructura de los datos a las necesidades particulares de cada aplicación.

Desde esta perspectiva, corresponde preguntarse cuáles registros se consultarán conjuntamente, qué atributos permitirán localizarlos y qué intervalos de información serán necesarios. Este razonamiento modifica la secuencia habitual del diseño: antes de decidir cuántas tablas resultan convenientes, se reconocen las consultas que el sistema deberá atender y la información que cada una necesita. En el caso de una red de sensores, buscar el historial de un dispositivo y recuperar las observaciones de una zona representan problemas relacionados, pero no equivalentes.

También adquiere relevancia la desnormalización,la cual se puede comprender como el mantenimiento de representaciones adicionales de una misma observación para atender consultas diferentes. La información puede repetirse, no por descuido, sino para facilitar ciertos accesos. Esta estrategia reduce la necesidad de reconstruir resultados a partir de múltiples estructuras; pero a cambio, aumentan el almacenamiento, las escrituras y la responsabilidad de mantener coherentes las distintas representaciones. Se trata, nuevamente, de una decisión que exige valorar costos y beneficios, no de una práctica inherentemente superior a la normalización.

II-D. Gestión de series temporales y particionamiento

Las series temporales reúnen observaciones asociadas con momentos específicos, de manera que su utilidad depende tanto de los valores registrados como de su evolución. En un sistema de monitoreo continuo, el historial aumenta de forma progresiva y debe seguir siendo posible recuperar lecturas recientes, comparar periodos e identificar cambios de comportamiento.

Pelkonen et al. [8] evidencian la importancia de adaptar el almacenamiento a las características de las series temporales. Su propuesta aprovecha regularidades en las marcas de tiempo y los valores para comprimir observaciones y facilitar el acceso a información reciente. Aunque este sistema no corresponde al mismo modelo utilizado en AquaSenseCR, ilustra por qué acumular mediciones no es suficiente si después no pueden recuperarse oportunamente.

Así, una estrategia para controlar el crecimiento del historial consiste en dividirlo en unidades temporales, como días o meses. Esta técnica, conocida habitualmente como *time bucketing*,

no elimina el crecimiento de los datos, sino que establece límites temporales para su agrupación. Resulta útil cuando las nuevas observaciones llegan con relativa regularidad y se conoce de antemano la forma en que serán consultadas.

Sin embargo, utilizar una referencia de tiempo como parte de la clave implica que la recuperación del historial de un dispositivo durante varios periodos requerirá identificar previamente cada intervalo involucrado. Por ello, es necesario equilibrar la cantidad de particiones con la información agrupada en cada una. Los intervalos amplios reúnen más observaciones, mientras que los pequeños obligan a consultar más particiones cuando se desea reconstruir periodos extensos. Le et al. [9] examinan el problema general de distribuir datos temporales mediante particiones de tamaño equilibrado, lo que aporta un fundamento para considerar cuidadosamente esta elección.

En aplicaciones con escrituras frecuentes también debe considerarse el riesgo de *hotspots*, es decir, concentraciones de actividad que pueden afectar ciertos recursos del sistema [10]. No basta con conocer cuántos registros contendrá cada partición, también interesa identificar cuáles recibirán operaciones de manera simultánea. La importancia de estas decisiones se aprecia en soluciones donde cientos de dispositivos producen observaciones durante periodos prolongados.

III. Tecnología seleccionada: ScyllaDB

III-A. Descripción y arquitectura

ScyllaDB es un sistema de gestión de bases de datos NoSQL, desarrollado en C++ y basado en el modelo de columnas anchas. Mantiene compatibilidad con *Cassandra Query Language* (CQL), lo que permite definir tablas y realizar operaciones de almacenamiento y consulta mediante una interfaz conocida para los sistemas compatibles con Apache Cassandra [11].

Una de sus principales particularidades es la arquitectura *shard-per-core*, en la cual cada núcleo administra una parte de los datos y de los recursos utilizados para procesar solicitudes. Esta organización se apoya en Seastar, un marco de ejecución asíncrona que busca reducir la competencia por recursos compartidos y aprovechar las capacidades del hardware [11]. Asimismo, ScyllaDB contempla el funcionamiento distribuido en varios nodos, aunque este último aspecto no forma parte del entorno experimental implementado.

La organización por núcleo debe distinguirse de la distribución entre servidores. La primera busca aprovechar los recursos internos de cada máquina, mientras que la segunda permite repartir información y solicitudes entre diferentes nodos. Así, una misma tecnología incorpora mecanismos para atender problemas de escalamiento distintos. No obstante, su aprovechamiento efectivo depende del número de núcleos y servidores disponibles, de la distribución de los registros y de la naturaleza de las consultas que recibe el sistema.

III-B. Mecanismos de almacenamiento y consulta

En ScyllaDB, la clave primaria se compone de una clave de partición y, cuando corresponde, de columnas de *clustering*. La primera permite localizar el conjunto de registros asociado con una partición; las segundas determinan su orden dentro de ella. Por esta razón, una consulta que conoce la clave de partición puede dirigirse hacia una porción específica de la información almacenada [12].

Este comportamiento refuerza la importancia del modelado orientado a consultas. Identificar una partición no representa la misma operación que buscar indiscriminadamente entre todas las observaciones almacenadas: la primera permite delimitar desde el inicio el conjunto relevante de registros. En cuanto al ordenamiento, las columnas de *clustering* también pueden favorecer la recuperación de intervalos sin reorganizar previamente la información.

De este forma, no basta con saber qué columnas componen una tabla: también interesa reconocer cuáles aparecen en las condiciones de búsqueda. Una consulta delimitada por identificador y mes puede dirigirse a una partición conocida y, dentro de ella, utilizar el orden temporal. Por el contrario, solicitar observaciones de todas las zonas sin identificar particiones exige una estrategia de recuperación diferente. Esta relación entre la pregunta y la clave constituye uno de los criterios con los que se evaluó la pertinencia del esquema elegido.

El motor dispone de mecanismos adicionales, como índices secundarios y vistas materializadas, pero su utilización requiere valorar los costos de mantenimiento y escritura [12]. En el presente trabajo se optó por tablas diseñadas explícitamente para cada patrón de recuperación. Esta decisión facilita vincular el comportamiento de las consultas con las claves seleccionadas, sin incorporar mecanismos adicionales cuya pertinencia tendría que evaluarse por separado.

III-C. Consistencia, replicación y escalabilidad

ScyllaDB permite establecer un factor de replicación para determinar cuántas copias de los datos se conservan en un clúster. También ofrece niveles de consistencia configurables, que indican cuántas réplicas deben responder a determinadas operaciones. Estas decisiones influyen en los compromisos entre disponibilidad, latencia y garantías de lectura y escritura [13], [14].

Por ejemplo, un nivel de consistencia `ONE` requiere la respuesta de una réplica, mientras que `QUORUM` exige la participación de una mayoría de las réplicas correspondientes. El efecto práctico de esta elección cambia según el factor de replicación configurado: mantener una sola copia no ofrece la misma capacidad de respuesta ante fallos que contar con varias réplicas en nodos diferentes. Así, las posibilidades de consistencia y disponibilidad deben entenderse en relación con la arquitectura utilizada, y no como propiedades garantizadas por el nombre del motor.

En términos de escalabilidad horizontal, la tecnología permite incorporar nodos y redistribuir información. Entre los mecanismos disponibles se encuentran los *tablets*, unidades que facilitan el movimiento y balanceo de datos en configuraciones que los utilizan [15]. Tales posibilidades describen las capacidades de la tecnología, no resultados comprobados en AquaSenseCR, cuyo despliegue se limitó a un nodo.

III-D. Pertinencia para AquaSenseCR

La selección de ScyllaDB responde a su capacidad para organizar información mediante claves de partición y atender consultas previamente identificadas. En el caso estudiado, interesa recuperar series temporales por dispositivo y por zona sin examinar indiscriminadamente todas las mediciones. También resulta útil mantener información derivada, como anomalías y resúmenes diarios, que pueda consultarse mediante estructuras específicas.

Sin embargo, su conveniencia no puede atribuirse únicamente al motor: una partición demasiado extensa o una distribución desequilibrada de operaciones también puede perjudicar el comportamiento de la solución [12]. A esto se suma el costo de almacenar una misma medición en diferentes tablas. Así, la elección de ScyllaDB proporciona mecanismos para resolver el problema, pero el resultado depende de cómo se utilicen. De ahí que el diseño físico constituya una parte fundamental del trabajo.

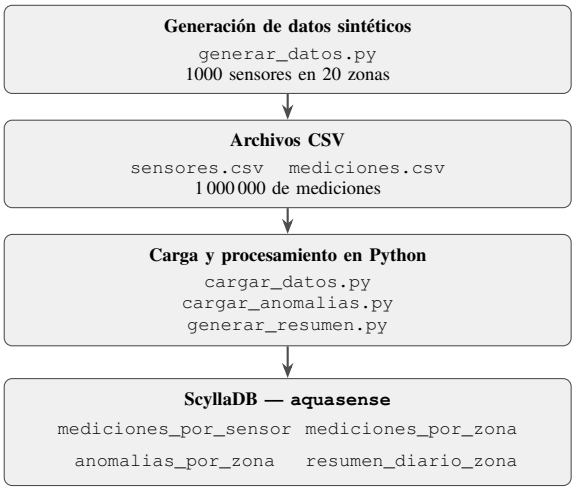


Figura 1. Arquitectura general de AquaSenseCR.

IV. Modelo y arquitectura

IV-A. Arquitectura general de la solución

AquaSenseCR se organizó en tres componentes: la generación reproducible de datos sintéticos mediante Python, su carga y almacenamiento en ScyllaDB, y las consultas utilizadas para examinar el funcionamiento de la base de datos. Esta separación facilita distinguir el origen de las observaciones de las operaciones necesarias para incorporarlas al modelo físico. También permite repetir la carga y las validaciones partiendo de los mismos archivos, aspecto importante cuando se estudia el comportamiento de una base de datos con fines experimentales.

El motor se ejecutó localmente mediante Docker Compose; los programas de Python se comunicaron con él a través de CQL y del puerto 9042. En consecuencia, la arquitectura representada corresponde a un entorno reproducible de generación y almacenamiento, no a una red de sensores físicos que envíe datos en tiempo real.

La Fig. 1 presenta el flujo principal, desde la producción de archivos CSV hasta su incorporación en las cuatro tablas del espacio de claves `aquasense`. Este esquema permite distinguir los procedimientos de generación y procesamiento de la organización lógica adoptada para almacenar los registros.

IV-B. Diseño físico orientado a consultas

El esquema comprende cuatro tablas, cada una asociada con necesidades particulares de recuperación. En `mediciones_por_sensor`, la clave primaria es `((sensor_id, mes_anio), timestamp)`: el identificador y el mes establecen la partición, mientras que la marca temporal organiza las lecturas en orden descendente. Así se facilita consultar el historial de un dispositivo y recuperar sus observaciones recientes.

En esta primera tabla, dos sensores distintos pueden compartir una misma marca temporal sin entrar en conflicto, ya que pertenecen a particiones diferentes. También las mediciones de un mismo sensor pueden separarse por mes sin perder su identificación original. La elección de un orden descendente responde a una pregunta especialmente relevante para el monitoreo: conocer qué ocurrió más recientemente antes de examinar, si fuera necesario, los valores anteriores. Así, una misma estructura permite atender lecturas recientes y rangos temporales dentro de un mes conocido.

La tabla `mediciones_por_zona` emplea la clave `((zona_id, mes_anio), timestamp, sensor_id)`, también ordenada de manera descendente por tiempo. Esta estructura permite localizar las observaciones de una zona y un mes determinados. Ambas tablas conservan las mismas mediciones, incluidos caudal, presión, temperatura, índice de calidad e indicador de anomalía; la duplicación responde a la necesidad de resolver consultas distintas sin combinar grandes conjuntos de registros.

El campo `sensor_id` cumple, además, una función diferenciada en la tabla por zona: aparece como segunda columna de *clustering* para distinguir las mediciones de distintos dispositivos registradas en una misma hora. Esta decisión es necesaria porque los 50 sensores de cada zona comparten las marcas temporales de la simulación. Sin ese identificador adicional en la clave, múltiples observaciones coincidirían en la misma posición lógica de almacenamiento. La estructura conserva así la individualidad de cada lectura sin renunciar a la recuperación por zona.

Por otra parte, `anomalias_por_zona` utiliza la partición `(zona_id, mes_anio)` y el ordenamiento por `timestamp` y `sensor_id`. Al conservar únicamente las lecturas clasificadas como anómalas, evita tener que examinarlas dentro del historial completo. Finalmente, `resumen_diario_zona` utiliza `zona_id` como clave de partición y `fecha` como clave de clustering descendente. Sus registros contienen conteos de mediciones y anomalías, presión máxima y promedios diarios de presión, caudal y calidad.

La diferencia entre ambas tablas derivadas debe destacarse. La de anomalías conserva observaciones individuales, de ma-

nera que todavía puede identificarse el sensor y el momento de una presión elevada. El resumen diario, en cambio, representa un nivel de agregación distinto: concentra en una fecha información que originalmente estaba repartida entre numerosas lecturas. Por ende, consultar el número de anomalías o la presión promedio de una zona no requiere volver a calcular esos indicadores sobre todas sus mediciones. La utilidad de cada tabla proviene, por tanto, de responder una pregunta operativa específica.

IV-C. Criterio de particionamiento temporal

La incorporación de `mes_anio` en las tablas de mediciones y anomalías permite separar el historial en particiones mensuales. Por ejemplo, las lecturas de enero y febrero de un mismo sensor no se acumulan bajo una única clave de partición. Esta decisión responde a la frecuencia de las observaciones simuladas y busca controlar el crecimiento histórico de los registros; cuando una consulta comprende varios meses, deben recuperarse las particiones correspondientes.

En el conjunto generado, cada sensor tiene 744 mediciones en enero y 256 en febrero. Por tanto, la clave formada por identificador y mes permite dividir sus 1000 lecturas en dos grupos temporales. En la tabla por zona, el mismo criterio agrupa las observaciones de 50 sensores, de manera que una zona concentra más registros por mes que un dispositivo individual. Esta diferencia ilustra por qué no basta con seleccionar una fecha como criterio de partición: debe considerarse también cuántos emisores de información comparten la misma clave.

En términos de diseño, la unidad elegida no representa simplemente una división cronológica: también expresa una decisión acerca del volumen de información que será consultado conjuntamente. Agrupar el historial por dispositivo favorece un seguimiento individual, mientras que hacerlo por zona reúne las observaciones de múltiples sensores para estudiar un mismo espacio geográfico. Por eso, aunque ambas tablas incorporan `mes_anio`, no resulta correcto asumir que sus particiones tendrán la misma cantidad de registros.

El caso de `resumen_diario_zona` es distinto, pues las fechas de cada zona se mantienen agrupadas para facilitar la consulta de sus indicadores diarios. En este sentido, el diseño prioriza las preguntas que debe responder cada tabla, aunque exige revisar la granularidad temporal si aumentan considerablemente la frecuencia de medición o el periodo de operación.

V. Implementación

V-A. Configuración del entorno

El proyecto se ejecutó mediante Docker Desktop y ScyllaDB 2026.1.14. El contenedor se configuró con un nodo, un núcleo de procesamiento, 2 GB de memoria y modo de desarrollo. Mediante Docker Compose se administró su ejecución y persistencia, mientras que los programas de Python utilizaron `scylla-driver` para establecer la conexión.

El espacio de claves `aquasense` se configuró con `NetworkTopologyStrategy` y factor de replicación uno. Después, el archivo `esquema.cql` permitió crear las cuatro tablas descritas previamente, manteniendo separada su definición del procedimiento de generación de datos.

Mantener estos archivos separados también permite reconstruir el entorno siguiendo una secuencia explícita: primero se inicia el servicio y se crea el esquema; después se generan los CSV y finalmente se ejecutan los programas de carga y verificación. Para un ejercicio académico, esta organización resulta relevante porque hace posible revisar las mismas operaciones sin depender de una base de datos compartida o de registros introducidos manualmente. En este caso, la conexión se realizó sobre la interfaz local del contenedor, sin incorporar servicios externos adicionales.

V-B. Generación de datos sintéticos

Dado que AquaSenseCR representa una empresa ficticia, se desarrolló `generar_datos.py` para construir un conjunto reproducible de observaciones. Se simularon 1000 sensores distribuidos uniformemente en 20 zonas, con 50 dispositivos por zona. Cada uno generó 1000 mediciones horarias a partir del 1 de enero de 2026; así, el historial se extendió hasta febrero y alcanzó un millón de lecturas.

Los registros contienen valores de caudal, presión, temperatura y un índice numérico de calidad, además de la marca temporal y los identificadores necesarios para localizar cada lectura. Para generar las variables se utilizó NumPy con una semilla aleatoria fija de 42. Este criterio no busca reproducir con exactitud el comportamiento hidráulico de una red particular, sino asegurar que el conjunto de datos pueda reconstruirse con los mismos valores y servir como base común para las pruebas.

La generación se planteó de modo que cada sensor contara con una secuencia temporal completa y regular. Esta uniformidad permitió conocer de antemano la cantidad esperada de observaciones por dispositivo y por zona, lo que más adelante facilitó las verificaciones de integridad. También posibilitó comparar consultas con horizontes temporales concretos: un intervalo de dos días para un sensor contiene menos registros que un día completo para los 50 dispositivos de una zona. Se trata de una simplificación deliberada que privilegia la evaluación del almacenamiento sobre la simulación detallada de fenómenos hidráulicos.

En particular, se tomó una presión de referencia de 3,5 , valor documentado en una zona de una red de distribución de agua estudiada por Patelis et al. [16]. Por su parte, Shammas y Al-Dhowalia [17] analizaron el vínculo entre presión y fugas mediante ensayos entre 1,5 y 5. Estos antecedentes permiten situar la variable de presión dentro de ejemplos reales de operación y estudio, sin establecer una condición universal aplicable a todas las redes.

Por lo tanto, se estableció para la simulación un **umbral experimental** de presión superior a 6,0, sin asumir que constituye un límite universal para las redes de agua potable. Se generaron 10 000 lecturas anómalas, equivalentes al 1 % del conjunto, y los datos se guardaron en `sensores.csv` y `mediciones.csv`.

V-C. Carga y procesamiento de la información

Mediante `cargar_datos.py` se insertaron las mediciones en las dos tablas principales, utilizando instrucciones CQL preparadas y operaciones concurrentes. Esta forma de ejecutar la carga permite procesar múltiples inserciones sin esperar a que cada una termine antes de iniciar la siguiente. El programa obtuvo el valor de `mes_anio` a partir de cada marca temporal, de manera que un millón de observaciones produjo dos millones de escrituras en las estructuras orientadas a sensor y zona.

El empleo de instrucciones preparadas permitió conservar una estructura uniforme para las inserciones y variar únicamente los valores de cada observación. Asimismo, el procesamiento concurrente respondió a la necesidad de incorporar una cantidad elevada de registros sin ejecutar todas las escrituras de manera estrictamente secuencial. La preparación de una misma lectura para dos tablas refleja el compromiso asumido en el diseño: realizar más operaciones durante la carga para disponer posteriormente de accesos diferenciados a la información.

Posteriormente, `cargar_anomalias.py` incorporó las 10 000 lecturas de presión anómala en la tabla correspondiente. Finalmente, `generar_resumen.py` calculó indicadores por día y zona, obteniendo 840 resúmenes correspondientes a 42 días y 20 zonas. El procedimiento conserva tanto las observaciones individuales como las tablas derivadas, destinadas a consultas operativas específicas.

VI. Pruebas

VI-A. Carga y recuperación de información

La evaluación comenzó con la carga masiva de las 1 000 000 de mediciones en las dos tablas principales. Se registró el tiempo total y se calculó la cantidad de observaciones procesadas por segundo, considerando el procedimiento completo de lectura del CSV, procesamiento mediante Python e inserción en ScyllaDB.

Posteriormente, `validaciones.py` ejecutó cinco patrones de consulta: lecturas recientes de un sensor, intervalo temporal por sensor, intervalo por zona, anomalías por zona y resúmenes diarios. Cada consulta se midió mediante 30 repeticiones, precedidas por cinco ejecuciones de calentamiento. Se registraron las filas recuperadas y se calcularon media, mediana, desviación estándar, mínimo, máximo y percentil 95 (P95) de la latencia.

Las consultas se seleccionaron para representar necesidades diferentes y no para ejecutar una única operación con cantidades variables de datos. En concreto, se solicitó un máximo de 20 lecturas recientes de un sensor y se recuperaron 48 observaciones para un intervalo de dos días del mismo dispositivo. Para la consulta por zona se utilizó un día completo, equivalente a 1200 observaciones. Las dos operaciones restantes consistieron en localizar 20 anomalías y recuperar 42 resúmenes diarios de una zona. Estos tamaños proporcionan un contexto necesario para interpretar después sus tiempos de respuesta.

VI-B. Verificación de registros y particiones

Además de medir los tiempos, se contrastaron los registros almacenados con las cantidades esperadas en las cuatro tablas. Se verificaron conteos por sensor y mes, por zona y mes, y de anomalías en sus particiones correspondientes. También se comprobó que los resúmenes diarios reprodujeran los totales de mediciones y anomalías y que las lecturas anómalas examinadas cumplieran el criterio de presión utilizado. Las comprobaciones se realizaron mediante claves de partición conocidas, revisando los meses de cada sensor y de cada zona.

Esta revisión era necesaria porque el almacenamiento no se realizó en una sola tabla. Una carga incompleta o una diferencia en la clasificación de anomalías podría dejar cantidades distintas entre las representaciones principales y derivadas. Por consiguiente, verificar tanto el millón de mediciones como la suma de sus agregados ofrece una comprobación adicional sobre la correspondencia entre los procedimientos ejecutados.

Se examinó, además, una muestra de lecturas clasificadas como anómalas para confirmar que su presión superara el umbral definido. Respecto a los resúmenes, no se verificó únicamente su existencia, también se contrastó que sus conteos acumulados representaran el millón de observaciones y las 10 000 anomalías previstas, además de comprobar la presencia de los indicadores promedio.

VI-C. Comparación de escrituras concurrentes

Como prueba complementaria se compararon dos patrones de escritura sobre una tabla auxiliar: 1000 inserciones distribuidas entre 20 zonas y otras 1000 concentradas en una sola zona. Ambos escenarios utilizaron concurrencia de 10 operaciones y se evaluaron mediante el tiempo total y las escrituras por segundo. Una vez finalizada la comparación, la tabla auxiliar fue eliminada. El propósito fue observar ambos comportamientos bajo una carga acotada, sin equiparar esta prueba con un diagnóstico completo de *hotspots*.

VII. Resultados e interpretación

VII-A. Carga masiva y rendimiento de consultas

La carga del millón de mediciones se completó en 199,04 segundos, con un promedio de 5 024,05 observaciones procesadas por segundo. Como cada registro se incorporó en dos tablas principales, la ejecución comprendió dos millones de operaciones de escritura. Esta cifra describe el procedimiento completo desarrollado en Python y constituye una referencia concreta del tiempo de incorporación para el volumen ensayado.

La Tabla I presenta las cinco consultas evaluadas. Cuatro registraron medianas inferiores a dos milisegundos, mientras que la consulta temporal por zona alcanzó 4,294 ms y recuperó el mayor volumen de información, con 1200 filas. La diferencia resulta compatible con la mayor cantidad de registros procesados, aunque las consultas no realizan operaciones idénticas y sus latencias no deben interpretarse como una comparación directa entre tablas.

Cabe destacar que la consulta por sensor se ejecutó sobre 48 observaciones, mientras que la consulta por zona abarcó 1200.

Aun cuando ambas se refieren a intervalos temporales, en el segundo caso el sistema debe entregar lecturas de distintos dispositivos que comparten una zona. De manera similar, la recuperación de las últimas 20 mediciones no busca reconstruir el historial completo, sino obtener una muestra reciente de un dispositivo. Estas diferencias ayudan a interpretar la tabla sin reducir la discusión a decidir cuál operación fue "mejor" a partir de su mediana.

Además de la mediana, se reportó el P95 para describir el comportamiento de las ejecuciones más lentas dentro de cada conjunto de repeticiones. Por ejemplo, en la consulta por zona este indicador fue de 5,488 ms, mientras que para las anomalías registró 1,435 ms. En consecuencia, la interpretación no se limita a un único tiempo promedio, sino que permite apreciar parte de la variabilidad de las operaciones ensayadas.

Tabla I
RENDIMIENTO DE CONSULTAS EN AQUASENSECR

Consulta	Filas	Mediana (ms)	P95 (ms)
Recientes por sensor	20	1,967	3,400
Intervalo por sensor	48	0,875	1,833
Intervalo por zona	1200	4,294	5,488
Anomalías por zona	20	0,907	1,435
Resumen diario	42	1,044	1,365

En particular, la recuperación de anomalías y resúmenes diarios registró medianas cercanas a un milisegundo. Esto muestra la utilidad práctica de mantener representaciones especializadas: las consultas pueden dirigirse hacia las observaciones o indicadores requeridos sin recalcularlos a partir del historial completo. La consulta de resúmenes recuperó 42 filas, correspondientes a los días cubiertos para una zona, mientras que la de anomalías obtuvo 20 registros en el caso evaluado. Así, ambas consultas representan necesidades distintas que fueron contempladas desde el diseño, y no adaptaciones introducidas después de cargar los datos.

En los resúmenes diarios esta decisión resulta clara: cada fila reúne los indicadores de numerosas mediciones para una combinación de zona y fecha. La consulta obtiene entonces resultados previamente calculados, en lugar de solicitar todas las observaciones y procesarlas nuevamente en Python. En el caso de las anomalías, el beneficio es distinto, pues permite conservar el detalle individual de las lecturas destacadas. La especialización de ambas estructuras ilustra dos maneras complementarias de convertir un historial amplio en información accesible para el monitoreo.

VII-B. Integridad y distribución temporal

Las verificaciones confirmaron 1 000 000 de registros en cada tabla principal, 10 000 anomalías y 840 resúmenes diarios. Asimismo, los conteos consolidados de estos últimos coincidieron con el millón de observaciones y las anomalías generadas, lo que respalda la coherencia de las distintas representaciones.

Respecto a la organización mensual, cada sensor presentó 744 registros en enero y 256 en febrero de 2026. Como cada zona contiene 50 sensores, se contabilizaron 37 200 y 12 800 mediciones por zona en esos mismos meses. Estos valores concuerdan con la frecuencia y el periodo definidos para la simulación, y permiten confirmar el funcionamiento lógico del particionamiento temporal. También ilustran que la cantidad de registros agrupados bajo una clave de zona es considerablemente mayor que la correspondiente a un sensor individual, aspecto que debe tomarse en cuenta al seleccionar la granularidad de las particiones.

VII-C. Escrituras concurrentes

El escenario distribuido entre 20 zonas alcanzó 7 198,86 escrituras por segundo, frente a 6 977,06 en el escenario concentrado en una sola zona. La diferencia observada, cercana al 3,2 %, favoreció ligeramente al primer escenario dentro de esta ejecución. Sin embargo, dado que cada patrón se probó una sola vez, el resultado se interpreta como una comparación exploratoria y no como evidencia definitiva de superioridad o ausencia de concentración de carga.

En conjunto, los resultados comprueban que AquaSenseCR pudo almacenar y consultar el volumen previsto, conservar sus representaciones derivadas y atender los patrones de acceso evaluados mediante particiones temporales identificables.

VIII. LIMITACIONES

El alcance de los resultados debe entenderse a partir de las condiciones del experimento. La implementación se ejecutó en un único nodo con factor de replicación uno, por lo que no fue posible comprobar empíricamente la escalabilidad multinodo, la redistribución entre servidores ni la tolerancia a fallos. De igual forma, las verificaciones de particionamiento se basaron en conteos de registros y no en mediciones de su tamaño físico o en un monitoreo directo de *hotspots*.

También debe considerarse que las mediciones son sintéticas y mantienen una frecuencia horaria uniforme. Aunque esta decisión asegura reproducibilidad, no reproduce toda la variabilidad de una red física; además, las anomalías se definieron únicamente a partir de la presión. Las pruebas de concurrencia se limitaron a una ejecución por escenario y los resultados de rendimiento corresponden a un volumen específico, de modo que no permiten anticipar por sí solos el comportamiento ante historiales o cargas superiores.

Finalmente, las tablas derivadas se alimentaron mediante programas ejecutados por etapas y no mediante un flujo automático de actualización continua. Una futura implementación operativa requeriría considerar la sincronización de estos registros conforme ingresen nuevas mediciones.

IX. CONCLUSIONES

El desarrollo de AquaSenseCR permitió implementar una solución reproducible de columnas anchas para un millón de mediciones sintéticas y comprobar su recuperación por sensor, zona e intervalo temporal. Asimismo, las anomalías y los resúmenes diarios demostraron la pertinencia de mantener estructuras diferenciadas para consultas que responden a necesidades operativas distintas. La coincidencia de los conteos originales con los registros derivados aporta, además,

evidencia de que las distintas etapas de generación, carga y procesamiento conservaron la información prevista.

Más allá del volumen almacenado, el principal aprendizaje del trabajo se relaciona con la importancia del diseño físico. Las claves compuestas y la división mensual del historial permitieron localizar conjuntos específicos de observaciones, mientras que las tablas derivadas redujeron la necesidad de recorrer el historial para recuperar anomalías o indicadores. Sin embargo, estas decisiones tienen un costo: duplicar información implica más escrituras y exige conservar coherencia entre las distintas representaciones.

Por tanto, la elección de ScyllaDB no supone una superioridad automática frente a los sistemas relacionales, sino una respuesta a las particularidades del problema planteado. La experiencia confirma que las ventajas de un modelo NoSQL dependen de su correspondencia con las consultas previstas y proporciona una base para futuras evaluaciones con mediciones reales, actualización continua y entornos distribuidos.

Repositorio de GitHub

Los códigos, las instrucciones de ejecución y las evidencias de las pruebas se encuentran disponibles en el **repositorio de GitHub de AquaSenseCR**, como apoyo a la reproducibilidad del trabajo.

Referencias

[1] B. W. Wade and D. D. Chamberlin, "IBM Relational Database Systems: The Early Years," *IEEE Annals of the History of Computing*, vol. 34, no. 4, pp. 38–48, 2012. doi: 10.1109/MAHC.2012.48.

[2] A. Ostfeld *et al.*, "The Battle of the Water Sensor Networks (BWSN): A design challenge for engineers and algorithms," *J. Water Resour. Plann. Manage.*, vol. 134, no. 6, pp. 556–568, 2008, doi: 10.1061/(ASCE)0733-9496(2008)134:6(556).

[3] G. A. Schreiner, D. Duarte, and R. dos S. Mello, "When Relational-Based Applications Go to NoSQL Databases: A Survey," *Information*, vol. 10, no. 7, Art. no. 241, 2019. doi: 10.3390/info10070241.

[4] R. Cattell, "Scalable SQL and NoSQL data stores," *ACM SIGMOD Rec.*, vol. 39, no. 4, pp. 12–27, 2011, doi: 10.1145/1978915.1978919.

[5] J. Han, H. E, G. Le, and J. Du, "Survey on NoSQL Database," in *2011 6th International Conference on Pervasive Computing and Applications (ICPCA)*, 2011, pp. 363–366. doi: 10.1109/ICPCA.2011.6106531.

[6] F. Chang *et al.*, "Bigtable: A distributed storage system for structured data," en *Proc. 7th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2006, pp. 205–218. [En línea]. Disponible: https://www.usenix.org/conference/osdi-06/bigtable-distributed-storage-system-structured-data.

[7] P. J. Sadalage and M. Fowler, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley Professional, 2012. [En línea]. Disponible en: https://martinfowler.com/books/nosql.html.

[8] T. Pelkonen *et al.*, "Gorilla: A fast, scalable, in-memory time series database," *Proc. VLDB Endow.*, vol. 8, no. 12, pp. 1816–1827, 2015, doi: 10.14778/2824032.2824078.

[9] W. Le, F. Li, Y. Tao y R. Christensen, "Optimal Splitters for Temporal and Multi-version Databases," en *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, pp. 109–120, 2013. doi: 10.1145/2463676.2465310.

[10] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, EE. UU.: O'Reilly Media, 2017.

[11] ScyllaDB, "ScyllaDB Shard-per-Core Architecture." [En línea]. Disponible: https://www.scylladb.com/product/technology/shard-per-core-architecture/.

[12] ScyllaDB, "Data Definition," ScyllaDB Documentation. [En línea]. Disponible: https://docs.scylladb.com/manual/stable/cql/ddl.html.

[13] ScyllaDB, "ScyllaDB Architecture – Fault Tolerance," ScyllaDB Documentation. [En línea]. Disponible: https://docs.scylladb.com/manual/stable/architecture/architecture-fault-tolerance.html.

[14] ScyllaDB, "Consistency Levels," ScyllaDB Documentation. [En línea]. Disponible: https://docs.scylladb.com/manual/stable/cql/consistency.html.

[15] ScyllaDB, "Data Distribution with Tablets," ScyllaDB Documentation. [En línea]. Disponible: https://docs.scylladb.com/manual/branch-2026.1/architecture/tablets.html.

[16] M. Patelis, V. Kanakoudis y A. Kravvari, "Pressure Regulation vs. Water Aging in Water Distribution Networks," *Water*, vol. 12, no. 5, art. 1323, 2020. doi: 10.3390/w12051323.

[17] N. K. Shammas y K. H. Al-Dhowalia, "Effect of Pressure on Leakage Rate in Water Distribution Networks," *Journal of King Saud University – Engineering Sciences*, vol. 5, no. 2, pp. 213–226, 1993. doi: 10.1016/S1018-3639(18)30581-6.